Research Plan & Chapter 4 Outline: Experimental Methods & Benchmarking Setup

Thesis Title: *Context is king - Retrieval-Augmented Generation in enterprise-ready systems*
Author: Antti-Jussi Asunmaa
Advisor: Lauri Salminen, MSc | **Supervisor:** Prof. Petri Vuorimaa
Target Chapter: Chapter 4 (*Research Methods & Experimental Setup*)

---

1. Executive Summary & Research Alignment

The objective of Chapter 4 is to establish a rigorous, reproducible, and enterprise-grade experimental methodology to evaluate four state-of-the-art Retrieval-Augmented Generation (RAG) optimization techniques:

1. **Small-to-Big Retrieval (M_1)**
2. **Two-Stage Semantic Re-ranking (M_2)**
3. **Prompt & Context Caching (M_3)**
4. **Hybrid Model Routing & Fallback Cascading (M_4)**

The benchmark suite is implemented on a production-grade Java 21 and Spring Boot 4.0 / Spring AI 2.0.0 stack integrated with Google Cloud Platform (GCP) Vertex AI services. The empirical research directly addresses four core Research Questions (RQs):

- **RQ1 (Latency Profile):** Reduction in Time to First Token (TTFT in ms), micro-phase processing breakdown, and end-to-end execution latency profiles (P50 median and P95 tail latency).
- **RQ2 (Operational Expenditure & Costs):** Financial cost reduction ($) and token consumption savings, taking into account model pricing tiers, prompt caching discounts (75% context caching discount), and model cascading savings.
- **RQ3 (Retrieval & Generation Quality):** Quality triad metrics evaluated via the offline Python RAGAS framework (*Context Precision*, *Context Recall*, *Faithfulness*, and *Answer Relevance*) using `gemini-3.1-pro` at $T=0.0$ as the primary evaluation judge. Address vendor self-preference bias through cross-validation options with alternative evaluation judges (e.g., GPT-4o, Claude 3.5 Sonnet, or open-weights models).
- **RQ4 (Combinatorial Stacking & Pareto Equilibrium):** Interactions, trade-offs, and friction between combined optimization techniques (e.g., re-ranking impacting cache hit rates), evaluated via Quality per Dollar (QPD) and Quality per Second (QPS) practical efficiency ratios across a full 16-condition experimental design matrix (C_0 to C_{15}) to establish the Pareto frontier.

---

2. Methodological Framework: Experimental Quantitative Benchmarking

This study adopts a two-stage **Experimental Quantitative Benchmarking** methodology to evaluate individual and combined optimization strategies under controlled enterprise conditions.

2.1 Two-Stage Experimental Strategy

1. **Single-Factor Isolated Testing:** Evaluate each optimization module (M_1, M_2, M_3, M_4) independently against the Naive Baseline (M_0) to quantify isolated performance gains and overhead.
2. **Combinatorial Stacking & Interaction Analysis:** Systematically evaluate pairwise, triple, and full-stack combinations ($M_1 + M_2 + M_3 + M_4$) to identify interactive friction, systemic trade-offs, and the Pareto optimal frontier (RQ4).

2.2 Execution Protocol

To eliminate network jitter, TLS handshakes, and JVM Just-In-Time (JIT) compilation cold-start bias, each query configuration is executed over **5 multi-run iterations**. The first run is discarded as a warm-up, and the remaining 4 runs are aggregated to compute statistical medians (P50), standard deviations, and P95 tail latencies.

2.3 Experimental System Architecture

```
flowchart TD
    subgraph Data Layer ["Data & Benchmark Layer"]
        A1["Primary Academic Benchmark: MuSiQue\n(100 sampled queries: 35% 2-hop,
65% 3/4-hop)"]
        A2["Secondary Corpus: Enterprise Support KB\n(500+ KB chunks + 200 synthetic
QA pairs)"]
    end

    subgraph SUT ["System Under Test (SUT) - Java 21 / Spring AI Orchestrator"]
        B["Spring Boot 4.0 / Spring AI 2.0.0\nBenchmark Execution Engine"]
        C1["Module M1: Small-to-Big\n(128-256 child -> 512-1024 parent)"]
        C2["Module M2: Semantic Re-ranking\n(GCP Vertex Ranking / TEI Fallback)"]
        C3["Module M3: Prompt Caching\n(GCP Gemini Context Cache >10k tokens)"]
        C4["Module M4: Model Cascading & Routing\n(Flash-Lite vs. Flash vs. Pro)"]

        B --> C1 & C2 & C3 & C4
    end

    subgraph Infrastructure ["Enterprise Infrastructure & GCP Services"]
        D1[("PostgreSQL 16 + pgvector\nDocker Local / Cloud SQL Production\n(MRL
768d, Cosine, HNSW Index)")]
        D2["GCP Vertex AI Gateway\n(IAM/ADC us-central1 / AI Studio Fallback)"]
        D3["3 Gemini Generative Model Tiers\ngemini-3.5-flash-lite | gemini-3.6-
flash | gemini-3.1-pro"]
        D4["Embeddings Architecture\nGCP gemini-embedding-001 (MRL 768d)"]
    end

    subgraph Metrics ["Metrics Ingestion & Evaluation Infrastructure"]
        E1["Spring Boot Actuator & Micrometer\n(Microsecond TTFT, Phase Latencies,
Token Counter)"]
        E2["Offline Python RAGAS Evaluator\n(Context Precision, Recall,
Faithfulness, Relevance)"]
        E3["Pareto & Cost Calculator\n(Exact $ OpEx, QPD, QPS Ratios)"]
    end

    Data Layer --> B
```

```
    SUT --> Infrastructure
    SUT --> Metrics
```

---

3. Technical Architecture & Enterprise GCP Stack

The System Under Test (SUT) is designed for production portability, code-level parity between local testing and cloud production, and enterprise governance:

- **Application Framework:** Java 21 LTS and **Spring Boot 4.0** with **Spring AI 2.0.0** (`spring-ai-starter-model-google-genai`).
- **Database Strategy & Local-to-Cloud Portability:**
  - *Engine:* **PostgreSQL 16** with the **`pgvector`** extension (v0.8.x).
  - *Local Development:* Containerized PostgreSQL 16 + `pgvector` in Docker for zero-cost, reproducible offline testing.
  - *GCP Production Deployment:* **Cloud SQL for PostgreSQL 16** with `pgvector` .
  - *Parity & Migration:* 100% SQL operator parity (`<=>` Cosine distance, HNSW indexing). Data schema migrated seamlessly via standard `pg_dump -Fc` and `pg_restore` .
- **GCP Vertex AI Gateway & Generative Model Tiers (M_4):**
  - *Enterprise Gateway:* GCP Vertex AI (`aiplatform.googleapis.com` in `us-central1` via IAM/ADC) with Google AI Studio API key fallback (`generativelanguage.googleapis.com`) for local developer environments.
  - *Model Tier 1 (Ultra-Fast / Routing):* `gemini-3.5-flash-lite` — Query intent routing, classification, and simple single-hop tasks.
  - *Model Tier 2 (Primary Workhorse):* `gemini-3.6-flash` — Medium-complexity context synthesis, fast generation, and coding tasks.
  - *Model Tier 3 (Flagship Reasoning):* `gemini-3.1-pro` — Complex multi-hop synthesis (3-hop and 4-hop queries) and fallback model.
- **Native Embeddings Architecture:**
  - *Primary Model:* GCP Native `gemini-embedding-001` (2,048 token window).
  - *Dimensionality Optimization:* Matryoshka Representation Learning (MRL) configured to output **768 dimensions** (`output_dimensionality=768`) to reduce vector store memory footprint and accelerate vector search without accuracy loss.
- **Two-Stage Re-ranking Architecture (M_2):**
  - *Production Standard:* GCP Vertex AI Ranking API (`semantic-ranker-default-004` and `semantic-ranker-fast-004`).
  - *Local Development Option:* TEI-hosted open cross-encoder microservice (`deberta-v3-large` or `jina-reranker-v2`).
  - *Workflow:* Vector retrieval selects top-200 candidate passages, which are re-sorted via joint cross-encoder self-attention down to top-k ($k=5$).
- **Instrumentation & Metrics Collection:** Spring Boot Actuator with Micrometer export for microsecond-level TTFT logging, phase latencies, and token consumption accounting.

---

4. Dataset & Benchmark Strategy

A hybrid dataset design provides both academic rigor and real-world enterprise relevance:

1. **Primary Academic Benchmark — MuSiQue Dataset (Trivedi et al., 2022):**
   - *Sample Size:* 100 multi-hop reasoning queries (35% 2-hop, 65% 3-hop/4-hop).

- *Rationale:* MuSiQue avoids single-hop reasoning shortcuts prevalent in older benchmarks like HotpotQA. Each query is paired with 20 paragraphs (~2,500–4,000 tokens), making it ideal for testing Small-to-Big retrieval (M_1), Semantic Re-ranking (M_2), and Prompt Caching (M_3).
- *Routing Replacement:* Hop-count difficulty tiers (2-hop vs. 3/4-hop) natively replace RouterBench for evaluating Model Cascading (M_4) routing between Gemini Flash and Pro tiers.

2. **Secondary Domain Corpus — Enterprise IT Customer Support:**
   - *Knowledge Base:* Technical documentation, SLA policies, and internal support guides (500+ document chunks).
   - *QA Set:* 200 synthetic query-answer pairs (40% single-hop informational, 40% multi-hop technical troubleshooting, 20% out-of-domain bypass).

---

5. Optimization Pipeline Implementations & Mechanics

Module	Optimization Strategy	Technical & Implementation Details
M_0	**Naive Flat Baseline**	Standard flat RAG: fixed 512-token chunks (50-token overlap), bi-encoder cosine top-5, static model, no caching.
M_1	**Small-to-Big Retrieval**	128–256 token child chunks indexed for dense vector search, expanding to 512–1024 token parent context windows upon retrieval via Spring AI metadata mapping.
M_2	**Two-Stage Semantic Re-ranking**	Bi-encoder candidate retrieval ($K_1=200$) pruned and re-ordered down to top-k ($k=5$) using GCP Vertex AI Ranking API (`semantic-ranker-default-004`/`fast-004`) or TEI cross-encoder.
M_3	**Prompt & Context Caching**	Dual-tier context caching: GCP Gemini Context Caching for static system prompts/contexts >10k tokens (75% token discount) + dynamic KV prefix reuse.
M_4	**Hybrid Model Routing & Fallback**	Native Spring AI ONNX classifier routing 2-hop queries to `gemini-3.6-flash` and 3/4-hop queries to `gemini-3.1-pro`, with fallback. Extension: Thinking Effort Routing / Reasoning Budget Control.

---

6. Factorial Experimental Design Matrix (Test Suite)

To systematically address **RQ4 (Interactions & Pareto Frontier)**, all 16 possible combinations (2^4 factorial design) are evaluated:

Condition ID	Configuration Code	Included Modules	Description
C_0	`Baseline`	Baseline (M_0)	Naive flat RAG (512-token chunks, static model, no re-ranking/caching)

C_1	`M1`	Small-to-Big (M_1)	Child chunk (128t) vector search $\rightarrow$ Parent window (512–1024t)
C_2	`M2`	Re-ranking (M_2)	Top-200 candidate retrieval $\rightarrow$ Vertex Ranking API top-5
C_3	`M3`	Caching (M_3)	GCP Context Caching (>10k static context)
C_4	`M4`	Cascading (M_4)	Query Intent Routing (`gemini-3.5-flash-lite` / `3.6-flash` / `3.1-pro`)
C_5	`M1+M2`	Small-to-Big + Re-ranking	Parent context expansion combined with semantic re-ranking
C_6	`M1+M3`	Small-to-Big + Caching	Expanded parent windows cached in GCP Context Cache
C_7	`M1+M4`	Small-to-Big + Cascading	Parent context expansion with model tier routing
C_8	`M2+M3`	Re-ranking + Caching	Evaluates re-ranking friction against context cache hit rates
C_9	`M2+M4`	Re-ranking + Cascading	Re-ranked context delivered to dynamically routed model tiers
C_{10}	`M3+M4`	Caching + Cascading	Context caching applied across routed Gemini model tiers
C_{11}	`M1+M2+M3`	Small-to-Big + Re-rank + Cache	Triple stack: Context expansion, re-ranking, and context caching
C_{12}	`M1+M2+M4`	Small-to-Big + Re-rank + Route	Triple stack: Context expansion, re-ranking, and model cascading
C_{13}	`M1+M3+M4`	Small-to-Big + Cache + Route	Triple stack: Context expansion, caching, and model cascading
C_{14}	`M2+M3+M4`	Re-rank + Cache + Route	Triple stack: Re-ranking, context caching, and model cascading
C_{15}	`M1+M2+M3+M4`	Full Stack	Complete enterprise optimization pipeline

Each condition is evaluated over **5 iterations** (1 warm-up run discarded, 4 recorded) across all benchmark queries.

---

7. Metrics & Measurement Infrastructure

7.1 Quantitative Efficiency Metrics (RQ1 & RQ2)

- **Time to First Token (TTFT):** Duration (ms) from request initiation until the first streaming response token frame is received.
- **End-to-End Latency:** Total request execution duration (ms).
- **Micro-Phase Latency Breakdown:** Individual timing logs for Vector Retrieval, Vertex Re-ranking, Context Cache lookup, LLM TTFT, and Stream Completion.
- **Operational Expenditure ($) Formula:** $$\text{Cost} = (N_{\text{uncached}} \cdot P_{\text{prompt}}) + (N_{\text{cached}} \cdot P_{\text{cached}}) + (N_{\text{completion}} \cdot P_{\text{completion}}) + (N_{\text{rerank_calls}} \cdot P_{\text{rerank}})$$ where P_{prompt}, P_{cached} (75% discount), $P_{\text{completion}}$, and P_{rerank} reflect official GCP Vertex AI pricing tiers.

7.2 Quality Metrics & Evaluation Judge Setup (RQ3)

Evaluated offline using Python **RAGAS (Es et al., 2024)**:

- **Context Precision:** Signal-to-noise ratio of relevant chunks in retrieved context.
- **Context Recall:** Ground-truth retrieval coverage.
- **Faithfulness:** Hallucination check quantifying groundedness against retrieved context.
- **Answer Relevance:** Semantic alignment between generated answer and user query.
- **Evaluation Judge Setup:** `gemini-3.1-pro` at $T=0.0$ as primary evaluation judge. Potential vendor self-preference bias is explicitly noted and addressed through cross-validation options with alternative judges (e.g., GPT-4o, Claude 3.5 Sonnet, or open-weights models).

7.3 Trade-off Analysis & Pareto Equilibrium (RQ4)

To evaluate the optimization return on investment (ROI) and establish the Pareto frontier, two practical efficiency ratios are computed:

- **Quality per Dollar (QPD):** $$\text{QPD} = \frac{\text{RAGAS Composite Score}}{\text{Total Request Cost in $}}$$
- **Quality per Second (QPS):** $$\text{QPS} = \frac{\text{RAGAS Composite Score}}{\text{End-to-End Latency in seconds}}$$

---

8. Proposed Section Structure for Chapter 4 (`04_research_methods.tex`)

```
\section{Research Methodology and Experimental Setup}
\label{sec:methods}

\subsection{Research Methodology}
\label{subsec:res_methodology}
% Details experimental quantitative benchmarking approach, 2-stage isolated &
stacking design, multi-run execution protocol, and factorial design matrix (C0 to
C15).

\subsection{System Implementation}
\label{subsec:sys_implementation}
% Locked tech stack decisions: Java 21 LTS, Spring Boot 4.0 / Spring AI 2.0.0,
PostgreSQL 16 + pgvector, GCP Vertex AI gateway, 3 Gemini generative model tiers,
gemini-embedding-001 (768d MRL), and two-stage re-ranking architecture.
```

```latex
\subsection{Optimization Pipeline}
\label{subsec:opt_pipeline}
% Mathematical & technical mechanics for Naive Baseline (M0), Small-to-Big (M1),
Two-Stage Re-ranking (M2), Prompt Caching (M3), Model Cascading & Routing (M4), and
Thinking Effort extension.

\subsection{Benchmark Datasets}
\label{subsec:bench_datasets}
% Academic MuSiQue benchmark (100 sampled multi-hop queries) + Enterprise IT
Customer Support KB (500+ chunks, 200 synthetic QA pairs).

\subsection{Evaluation Setup \& Metrics}
\label{subsec:eval_metrics}
% Quantitative efficiency metrics (TTFT, micro-phase latencies, OpEx cost equation),
RAGAS quality triad metrics, judge setup & bias mitigation, and trade-off ratios
(QPD, QPS).
```

---

9. Advisor Presentation & Verification Checklist

> *[!TIP]* ***Key Talking Points for Advisor Meetings:***
>
> 1. ***Why Java 21 / Spring Boot 4.0 / Spring AI 2.0.0 + GCP Vertex AI?*** *Demonstrates enterprise production readiness within standard JVM enterprise architectures, contrasting with python-centric academic scripts.*
> 2. ***Why PostgreSQL 16 +*** `pgvector` ***with Matryoshka Representation Learning (MRL)?*** *Achieves 768-dimensional storage optimization while maintaining 100% parity between local Docker benchmarking and GCP Cloud SQL production deployment.*
> 3. ***Why MuSiQue over HotpotQA/RouterBench?*** *MuSiQue's 2-hop vs. 3/4-hop queries (~2.5k–4k tokens per query) eliminate single-hop shortcuts and natively evaluate Model Cascading (M_4) routing while stressing M_1, M_2, M_3.*
> 4. ***How does RQ4 bring academic novelty?*** *Prior literature evaluates M_1–M_4 in isolation. This thesis quantifies combinatorial interaction friction (e.g., does semantic re-ranking degrade context caching hit rates?) across all 16 conditions on the Pareto frontier.*